МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
Кафедра «Интеллектуальные информационные технологии»

ЛАБОРАТОРНАЯ РАБОТА №2

по дисциплине «Основы машинного обучения»
Тема: «Линейные модели для задач регрессии и классификации»

Выполнил:
студент 2 курса
группы АС-67
Швед М. С.

Проверил:
Крощенко А. А.

Брест 2026

Цель: изучить применение линейной и логистической регрессии для решения практических задач. Научиться обучать модели, оценивать их качество с помощью соответствующих метрик и интерпретировать результаты.

Общее задание: выполнить задания по варианту (регрессия и классификация), построить все требуемые визуализации и рассчитать метрики, написать отчет, создать пул-реквест в репозиторий с кодом решения и отчетом в формате pdf.

Используемые инструменты: Python, Pandas, NumPy, Matplotlib, scikit-learn, Orange Data Mining.

Задания по вариантам:
Вариант 1

• Регрессия (Прогнозирование стоимости жилья в Калифорнии)
1. California Housing
2. Предсказать медианную стоимость дома (median_house_value)
3. Задания:
   ▪ загрузите данные и разделите их на обучающую и тестовую выборки;
   ▪ обучите модель линейной регрессии на обучающих данных;
   ▪ сделайте предсказания для тестовой выборки;
   ▪ оцените качество модели, рассчитав метрики MSE (Mean Squared Error) и R^2 (Coefficient of Determination);
   ▪ визуализируйте результат: постройте диаграмму рассеяния для признака median_income (медианный доход) и целевой переменной, нанеся на неё линию регрессии.

• Классификация (Прогнозирование выживаемости на «Титанике»)
1. Titanic
2. Предсказать, выжил ли пассажир (Survived)
3. Задания:
   ▪ загрузите и предварительно обработайте данные (заполните пропуски, преобразуйте категории в числа);
   ▪ обучите модель логистической регрессии;
   ▪ оцените качество модели, рассчитав Accuracy, Precision и Recall;
   ▪ постройте и проанализируйте матрицу ошибок (confusion matrix).

Код:

```
import os
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.metrics import (mean_squared_error, r2_score, accuracy_score,
                             precision_score, recall_score, confusion_matrix,
                             ConfusionMatrixDisplay)
from sklearn.preprocessing import StandardScaler


BASE_DIR = os.path.dirname(os.path.abspath(__file__))
RANDOM_STATE = 42



def regression_task():
    print("=" * 60)
    print("РЕГРЕССИЯ: стоимость жилья в Калифорнии (California Housing)")
    print("=" * 60)


    try:
        df = pd.read_csv(os.path.join(BASE_DIR, 'california_housing.csv'))
    except FileNotFoundError:
        print("Ошибка: файл 'california_housing.csv' не найден.")
        return


    print(f"Размер датасета: {df.shape}")
    print(f"Пропуски:\n{df.isnull().sum()[df.isnull().sum() > 0]}")


    # Заполняем пропуски в total_bedrooms медианой
    df['total_bedrooms'] = df['total_bedrooms'].fillna(df['total_bedrooms'].median())
    # Категориальный признак ocean_proximity -> one-hot кодирование
    df = pd.get_dummies(df, columns=['ocean_proximity'], drop_first=True, dtype=int)


    target = 'median_house_value'
    X = df.drop(columns=[target])
    y = df[target]


    # Разделение на обучающую (80%) и тестовую (20%) выборки
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=RANDOM_STATE)
    print(f"Обучающая выборка: {X_train.shape[0]}, тестовая: {X_test.shape[0]}")


    # Обучение модели линейной регрессии на всех признаках
    model = LinearRegression()
    model.fit(X_train, y_train)


    # Предсказания для тестовой выборки
    y_pred = model.predict(X_test)


    mse = mean_squared_error(y_test, y_pred)
    r2 = r2_score(y_test, y_pred)
    print("\nМодель на всех признаках:")
    print(f"MSE  (среднеквадратичная ошибка): {mse:,.2f}")
    print(f"RMSE (корень из MSE):             {np.sqrt(mse):,.2f}")
    print(f"R2   (коэффициент детерминации):  {r2:.4f}")


    print("\nКоэффициенты модели:")
    coefs = pd.Series(model.coef_, index=X.columns).sort_values(key=abs,
ascending=False)
    for name, value in coefs.items():
        print(f"  {name:28s} {value:14,.2f}")
```

```
    # Парная регрессия median_house_value ~ median_income (для линии на графике)
    model_income = LinearRegression()
    model_income.fit(X_train[['median_income']], y_train)
    y_pred_income = model_income.predict(X_test[['median_income']])
    mse_income = mean_squared_error(y_test, y_pred_income)
    r2_income = r2_score(y_test, y_pred_income)
    print("\nМодель только на median_income:")
    print(f"y = {model_income.coef_[0]:,.2f} * median_income +
{model_income.intercept_:,.2f}")
    print(f"MSE: {mse_income:,.2f}, RMSE: {np.sqrt(mse_income):,.2f}, R2:
{r2_income:.4f}")


    # Диаграмма рассеяния median_income vs median_house_value с линией регрессии
    plt.figure(figsize=(10, 6))
    plt.scatter(X_test['median_income'], y_test, alpha=0.3, s=10,
                color='steelblue', label='Тестовые данные')
    x_line = np.linspace(X['median_income'].min(), X['median_income'].max(), 100)
    y_line = model_income.predict(pd.DataFrame({'median_income': x_line}))
    plt.plot(x_line, y_line, color='red', linewidth=2,
             label=f'Линия регрессии (R² = {r2_income:.3f})')
    plt.title('Зависимость стоимости жилья от медианного дохода')
    plt.xlabel('median_income (медианный доход, десятки тыс. $)')
    plt.ylabel('median_house_value (медианная стоимость дома, $)')
    plt.legend()
    plt.grid(True, linestyle='--', alpha=0.6)
    plt.tight_layout()
    plt.savefig(os.path.join(BASE_DIR, 'regression_plot.png'), dpi=150)
    plt.show()


    # Дополнительно: предсказанные значения против фактических (модель на всех
признаках)
    plt.figure(figsize=(7, 7))
    plt.scatter(y_test, y_pred, alpha=0.3, s=10, color='seagreen')
    lims = [y_test.min(), y_test.max()]
    plt.plot(lims, lims, 'r--', linewidth=2, label='Идеальное предсказание')
    plt.title(f'Предсказание vs факт (все признаки, R² = {r2:.3f})')
    plt.xlabel('Фактическая стоимость, $')
    plt.ylabel('Предсказанная стоимость, $')
    plt.legend()
    plt.grid(True, linestyle='--', alpha=0.6)
    plt.tight_layout()
    plt.savefig(os.path.join(BASE_DIR, 'regression_pred_vs_true.png'), dpi=150)
    plt.show()



def classification_task():
    print("\n" + "=" * 60)
    print("КЛАССИФИКАЦИЯ: выживаемость на Титанике (Titanic)")
    print("=" * 60)


    try:
        df = pd.read_csv(os.path.join(BASE_DIR, 'titanic.csv'))
    except FileNotFoundError:
        print("Ошибка: файл 'titanic.csv' не найден.")
        return


    print(f"Размер датасета: {df.shape}")
    print(f"Пропуски до обработки:\n{df.isnull().sum()[df.isnull().sum() > 0]}")


    # Удаляем признаки, не несущие полезной информации для модели
    # (Cabin пропущен у 77% пассажиров)
    df = df.drop(columns=['PassengerId', 'Name', 'Ticket', 'Cabin'])


    # Заполнение пропусков
    df['Age'] = df['Age'].fillna(df['Age'].median())
    df['Embarked'] = df['Embarked'].fillna(df['Embarked'].mode()[0])
```

```
    print(f"Пропусков после обработки: {df.isnull().sum().sum()}")

    # Преобразование категорий в числа
    df['Sex'] = df['Sex'].map({'male': 0, 'female': 1})
    df = pd.get_dummies(df, columns=['Embarked'], drop_first=True, dtype=int)

    X = df.drop(columns=['Survived'])
    y = df['Survived']

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=RANDOM_STATE, stratify=y)
    print(f"Обучающая выборка: {X_train.shape[0]}, тестовая: {X_test.shape[0]}")

    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)

    # Обучение логистической регрессии
    model = LogisticRegression(max_iter=1000, random_state=RANDOM_STATE)
    model.fit(X_train_scaled, y_train)

    y_pred = model.predict(X_test_scaled)

    accuracy = accuracy_score(y_test, y_pred)
    precision = precision_score(y_test, y_pred)
    recall = recall_score(y_test, y_pred)
    print(f"\nAccuracy:  {accuracy:.4f}")
    print(f"Precision: {precision:.4f}")
    print(f"Recall:    {recall:.4f}")

    print("\nКоэффициенты модели (на стандартизированных признаках):")
    coefs = pd.Series(model.coef_[0], index=X.columns).sort_values(key=abs,
ascending=False)
    for name, value in coefs.items():
        print(f"  {name:12s} {value:8.4f}")

    # Матрица ошибок
    cm = confusion_matrix(y_test, y_pred)
    tn, fp, fn, tp = cm.ravel()
    print("\nМатрица ошибок:")
    print(cm)
    print(f"TN (верно: не выжил) = {tn}")
    print(f"FP (ошибочно: выжил) = {fp}")
    print(f"FN (ошибочно: не выжил) = {fn}")
    print(f"TP (верно: выжил) = {tp}")

    disp = ConfusionMatrixDisplay(confusion_matrix=cm,
                                  display_labels=['Не выжил (0)', 'Выжил (1)'])
    disp.plot(cmap=plt.cm.Blues)
    plt.title('Матрица ошибок (Confusion Matrix)')
    plt.xlabel('Предсказанный класс')
    plt.ylabel('Истинный класс')
    plt.tight_layout()
    plt.savefig(os.path.join(BASE_DIR, 'confusion_matrix.png'), dpi=150)
    plt.show()



if __name__ == "__main__":
    regression_task()
    classification_task()
```

Результаты выполнения программы:

1. Регрессия (California Housing)

Датасет содержит 20 640 записей и 10 столбцов. В признаке total_bedrooms 207 пропусков - они заполнены медианой. Категориальный признак ocean_proximity преобразован one-hot кодированием. Данные разделены на обучающую (16 512) и тестовую (4 128) выборки в соотношении 80/20.

```
============================================================
РЕГРЕССИЯ: стоимость жилья в Калифорнии (California Housing)
============================================================
Размер датасета: (20640, 10)
Пропуски:
total_bedrooms    207
dtype: int64
Обучающая выборка: 16512, тестовая: 4128


Модель на всех признаках:
MSE  (среднеквадратичная ошибка): 4,908,476,721.16
RMSE (корень из MSE):             70,060.52
R2   (коэффициент детерминации):  0.6254


Коэффициенты модели:
  ocean_proximity_ISLAND           136,125.07
  ocean_proximity_INLAND           -39,786.66
  median_income                     39,473.98
  longitude                        -26,838.27
  latitude                         -25,468.35
  ocean_proximity_NEAR BAY          -5,136.64
  ocean_proximity_NEAR OCEAN         3,431.14
  housing_median_age                 1,102.19
  total_bedrooms                       102.79
  households                            48.25
  population                           -38.17
  total_rooms                           -6.02


Модель только на median_income:
y = 41,933.85 * median_income + 44,459.73
MSE: 7,091,157,771.77, RMSE: 84,209.01, R2: 0.4589
```

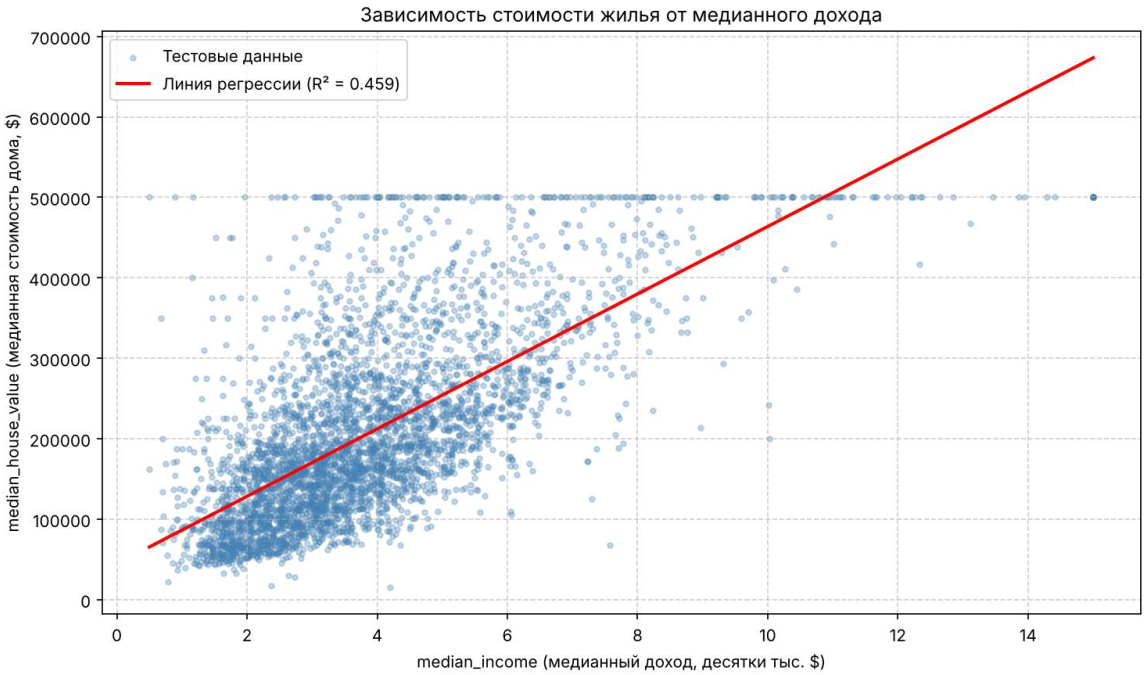


Рисунок 1 - Диаграмма рассеяния median_income - median_house_value с линией регрессии

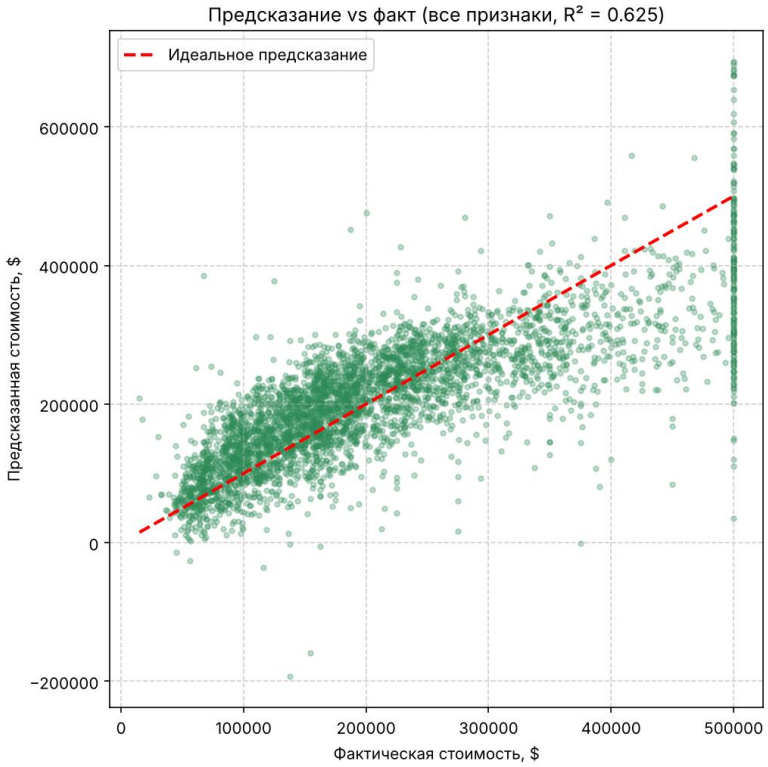


Рисунок 2 - Предсказанные и фактические значения (модель на всех признаках)

Модель линейной регрессии на всех признаках объясняет около 62,5 % дисперсии стоимости жилья ($R^2 = 0{,}6254$), MSE $\approx 4{,}91 \cdot 10^9$ (RMSE $\approx 70\,061$ \$, т.е. средняя ошибка порядка 70 тыс. \$). Наиболее значимый числовой признак - median_income: при росте медианного дохода на 1 (10 тыс. \$) стоимость дома растёт в среднем на ~39,5 тыс. \$. Парная регрессия только по median_income даёт $R^2 = 0{,}4589$ - доход сам по себе объясняет почти половину дисперсии. На

графике видна горизонтальная «полка» на уровне 500 000 $: значения целевой переменной в датасете обрезаны сверху, из-за чего линейная модель систематически ошибается для дорогих домов.

2. Классификация (Titanic)

Датасет содержит 891 запись. Пропуски: Age - 177 (заполнены медианой), Embarked - 2 (заполнены модой), Cabin - 687 (столбец удалён, т.к. пропущено 77 % значений). Также удалены PassengerId, Name, Ticket. Признак Sex закодирован 0/1, Embarked - one-hot кодированием. Признаки стандартизированы (StandardScaler), разбиение 80/20 со стратификацией по Survived.

```
Размер датасета: (891, 12)
Пропуски до обработки:
Age         177
Cabin       687
Embarked      2
dtype: int64
Пропусков после обработки: 0
Обучающая выборка: 712, тестовая: 179


Accuracy:  0.8045
Precision: 0.7931
Recall:    0.6667


Коэффициенты модели (на стандартизированных признаках):
  Sex            1.2696
  Pclass        -0.9287
  Age           -0.5029
  SibSp         -0.2627
  Embarked_S    -0.1718
  Fare           0.0996
  Embarked_Q     0.0807
  Parch         -0.0672


Матрица ошибок:
[[98 12]
 [23 46]]
TN (верно: не выжил) = 98
FP (ошибочно: выжил) = 12
FN (ошибочно: не выжил) = 23
TP (верно: выжил) = 46
```

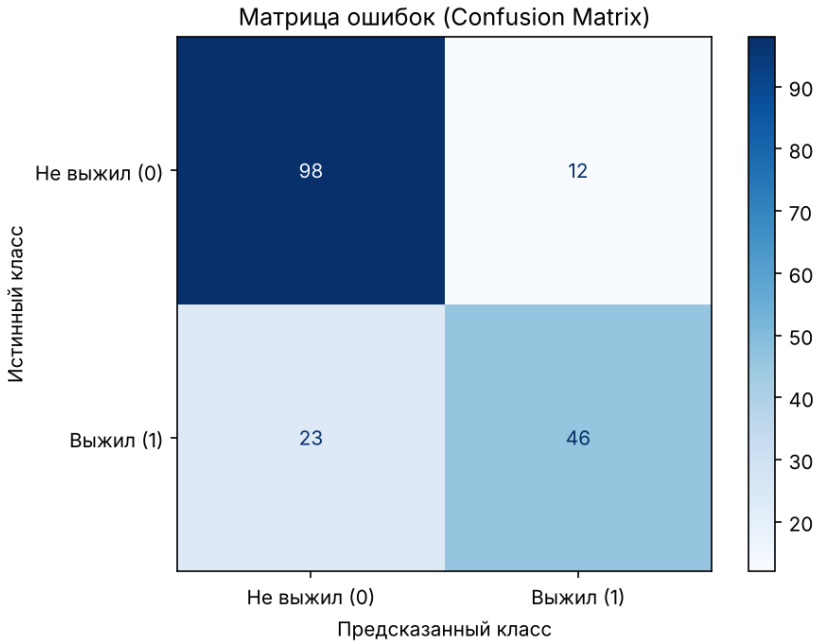


Рисунок 3 - Матрица ошибок логистической регрессии

Анализ матрицы ошибок. Из 179 пассажиров тестовой выборки верно классифицировано 144 (98 TN + 46 TP), Accuracy = 0,8045. Модель лучше распознаёт погибших: из 110 погибших верно определены 98 (ошибочно «выжившими» названы 12 - FP). Из 69 выживших верно определены только 46, а 23 модель ошибочно отнесла к погибшим (FN). Поэтому Precision = 46/(46+12) = 0,7931 - если модель говорит «выжил», она права в 79 % случаев, а Recall = 46/(46+23) = 0,6667 - модель находит лишь 2/3 реально выживших. Основной тип ошибки - ложноотрицательные (FN), что связано с дисбалансом классов (выжили только 38 % пассажиров). Наибольшее влияние на выживание оказывают пол (женщины выживали чаще), класс каюты (у 1-го класса шансы выше) и возраст (дети выживали чаще).